

Tiling Game

Barchin og Charos spiller et spill på et $2N \times 2M$ rutenett bestående av 1×1 ruter. Radene er nummerert fra 0 til $2N - 1$ fra topp til bunn, og kolonnene nummerert fra 0 til $2M - 1$ fra venstre til høyre. For $0 \leq i < 2N$ og $0 \leq j < 2M$, ruten i rad i og kolonne j har koordinat (i, j) .

Barchin gir Charos $N \cdot M$ **blokker**, en etter en. Hver blokk er et 2×2 kvadrat bestående av fire 1×1 **fliser**. Barchin har fargelagt hver flis av en blokk enten svart eller hvit, men garanterer at hver blokk har **minst en hvit flis**.

Charos må plassere hver blokk i rutenettet **før hun får neste blokk**. Blokker kan ikke roteres. Hver blokk må dekke akkurat 4 ruter. Videre må topp-venstre flis av hver blokk dekke en rute med både partalls kolonne og partalls rad koordinater. Hver rute i rutenettet kan dekkes av høyst en blokk.

Barchin vinner hvis det etter en blokk er plassert eksisterer et 2×2 kvadrat hvor alle rutene er dekket av svarte fliser. Formelt, hvis alle rutene (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ er dekket av svarte ruter for en $0 \leq a < 2N - 1$ og $0 \leq b < 2M - 1$, vinner Barchin. a og b trenger ikke være partall.

Charos vinner hvis hun plasserer alle $N \cdot M$ blokker uten at Barchin vinner. Merk at å plassere $N \cdot M$ blokker vil dekke hele rutenettet.

Oppgaven din er å implementere en strategi hvor Charos alltid vinner. Det kan vises at, under de gitte begrensningene har Charos alltid en vinnende strategi, uansett hvordan blokkene er fargelagt.

Implementasjonsdetaljer

Du skal implementere to funksjoner:

```
void init(int N, int M)
```

- N : halvparten antall rader i rutenettet.
- M : halvparten antall kolonner i rutenettet.
- Funksjonen kalles nøyaktig en gang per testcase, i starten av programmet ditt.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : henholdsvis fargene av øvre venstre, øvre høyre, nedre venstre, nedre høyre fliser av blokken du får som vist i figuren under. Hver verdi er enten 0 (hvit) eller 1 (svart).
- Denne funksjonen kalles nøyaktig $N \cdot M$ ganger per testcase, etter `init`.



Funksjonen skal returnere et par av heltall (i, j) , hvor i er raden og j er kolonnen av øvre venstre flis av blokken plasseres i rutenettet. Både i og j må være partall, og 2×2 kvadratet dekket av blokken kan ikke overlappe en tidligere plassert blokk.

Hvis `receive_block` returnerer et par som ikke tilfredstiller disse kravene, eller hvis du danner en 2×2 kvadrat av ruter som er bare svart, vil graderen terminere programmet ditt og resultatet av test casen vil være `Output isn't correct`.

Graderen er **ikke adaptiv**. Det betyr at rekkefølgen av blokker er forhåndsbestemt.

Begrensninger

For hver blokk, la S være antall svarte fliser i en blokk. Altså, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ for hver blokk.

Deloppgaver

Deloppgave	Poeng	Ekstra begrensninger
1	6	$S = 1$ for hver blokk, og $N = 2$.
2	16	$S = 3$ for hver blokk. $N = M$, N er partall, og hver av de fire mulige fargeleggingene dukker opp nøyaktig $\frac{N^2}{4}$ ganger.
3	10	$S = 1$ for hver blokk.
4	29	$S \leq 2$ for hver blokk.
5	39	Ingen ekstra begrensninger.

Eksempel

Gitt et spill med $N = 1$ og $M = 2$, så rutenettet har 2 rader og 4 kolonner. Graderen kaller først:

```
init(1, 2)
```

Til å begynne med er alle rutene tomme. Rutenettet ser slik ut.

	0	1	2	3
0				
1				

Det er $N \cdot M = 2$ blokker som skal plasseres. Anta at Barchin gir en blokk med tre sorte fliser og en hvit flis topp-høyre. Graderen kaller:

```
receive_block(1, 0, 1, 1)
```

Charos plasserer blokken på venstre side ved å returnerer $(0, 0)$.

Rutenettet ser nå slik ut:

	0	1	2	3
0				
1				

Barchin gir så en blokk med tre svarte fliser og en hvit flis top-venstre:

```
receive_block(0, 1, 1, 1)
```

Den eneste gyldige plasseringen med partalls koordinater er $(0, 2)$, så Charos returnerer $(0, 2)$. Rutenettet blir sende ut slik:

	0	1	2	3
0				
1				

Ingen 2×2 kvadrat er dekket av bare sorte fliser, så Charos vinner spillet.

Sample Grader

Input format:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Output format:

```
R[0] C[0]
R[1] C[1]
...
R[NM-1] C[NM-1]
```

Her er $R[k]$ og $C[k]$ er paret av koordinater det k -ende kallet til `receive_block` returnerer.